

Enabling External PSRAM for Zephyr Applications on ESP32 SoCs

AUTHOR
Marcio Ribeiro
Embedded Software Engineer at Espressif

> ***This article presents ideas and suggestions on different scenarios to use the PSRAM available in some Espressif devices.***

Although ESP32 SoCs typically feature a few hundred kilobytes of internal SRAM, some embedded IoT applications—such as those featuring sophisticated graphical user interfaces, handling large data volumes, or performing complex data processing—may require much more RAM. To meet these demands, Espressif provides modules that feature external PSRAM.

In this article, you will learn how to configure Zephyr to enable your application to take full advantage of the PSRAM available on ESP32 modules. We will explore three different strategies for utilizing this PSRAM: using it for dynamic memory allocation, adding it to your ESP32 memory map, and running code from PSRAM instead of Flash.

Getting Started

memory. It is a type of dynamic random-access memory (DRAM) that includes additional circuitry to handle memory cell refresh automatically, while DRAM requires an external memory controller to refresh the cell memory charge. This additional circuitry allows PSRAM to keep the cost close to DRAM but provides ease of use and high speed similar to static random-access memory (SRAM).

PSRAM is typically accessible via high-speed serial buses, and some ESP32 series SoCs can communicate with PSRAM and map its contents into the CPU's virtual memory space through a memory management unit (MMU).

In a simplified way, we can describe the relationship among PSRAM, the MMU, and the CPU as follows:

- Data is automatically copied from PSRAM to the data cache when the CPU attempts to access a memory position not currently in the cache.
- Conversely, when CPU modifies data in the cache, the updated contents are copied back to PSRAM.

At the time of writing, while many ESP32 SoCs support external PSRAM, not all of them are supported on Zephyr yet.

SoC series	PSRAM capable	Currently supported on Zephyr
ESP32	yes	yes
ESP32-S2	yes	yes
ESP32-S3	yes	yes
ESP32-C5	yes	no
ESP32-C61	yes	no
ESP32-P4	yes	no

What Are ESP32 Modules and PSRAM?

name is ESP32-S3-WROOM-1-N8R8

Field	Contents	Meaning
SoC series	ESP32-S3	ESP32-S3 SoC series
flash size	N8	8 MB
PSRAM size	R8	8 MB

If a module does not contain PSRAM, the R field is absent.

Additional resources for deciphering product nomenclatures can be found in a chip datasheet and module datasheet of a respective series. For example, see the ESP32-S3 series chip datasheet and module datasheet.

Using PSRAM with Custom ESP32 Hardware

Although Espressif offers modules featuring PSRAM across various ESP32 SoC series, there may be cases where custom hardware must be developed to meet specific requirements, such as PSRAM capacity or the SPI interface bus width, which impacts the time for each memory operation and overall data access speed. Additionally, there may be other reasons unrelated to PSRAM that necessitate custom hardware.

In this case, PSRAM will share data I/O and clock lines with external flash memory, but not chip-select line. PSRAM and flash have separate chip select lines (for example, see the ESP32-S3 Chip Datasheet). For information on how to connect off-package PSRAM lines, the respective SoC datasheet must be consulted.

The following SPI modes are supported to access external PSRAM memories:

- Single SPI
- Dual SPI
- Quad SPI

- OPI

There are limits regarding the maximum PSRAM capacity that each ESP32 SoC series can manage. As an example, ESP32-S3 can manage PSRAMs up to 32 MB.

Espressif currently offers some PSRAM models:

Part number	Memory Capacity	Operating Voltage
ESP-PSRAM32	4 MB	1.8V
ESP-PSRAM32H	4 MB	3.3V
ESP-PSRAM64	8 MB	1.8V
ESP-PSRAM64H	8 MB	3.3V

How to Set Up PSRAM in Zephyr

The Kconfig parameters related to PSRAM usage in Zephyr can be found in zephyr/soc/espressif/commom/Kconfig.spiram

Below are the main parameters related to PSRAM, along with a brief description of each:

ESP_SPIRAM: This configuration parameter enables support for an external SPI RAM chip, connected in parallel with the main SPI flash chip. If enabled, it automatically enables SHARED_MULTI_HEAP.

ESP_SPIRAM_HEAP_SIZE: This configuration parameter specifies SPIRAM heap size.

ESP_SPIRAM_MEMTEST: This configuration parameter controls SPIRAM memory test during initialization. It is enabled by default and can be disabled for faster startup.

Please note that SPIRAM_MODE_OCT is only available in ESP32-S3.

SPIRAM_TYPE: This configuration parameter defines the type of SPIRAM chip in use:

SPIRAM_SPEED: This configuration parameter sets the SPIRAM clock speed in MHz:

Value	Clock speed
SPIRAM_SPEED_20M	20 MHz
SPIRAM_SPEED_26M	26 MHz
SPIRAM_SPEED_40M	40 MHz
SPIRAM_SPEED_80M	80 MHz
SPIRAM_SPEED_120M	120 MHz

SPIRAM_FETCH_INSTRUCTIONS: This configuration parameter allows moving instructions from flash to PSRAM. If enabled, instructions in flash will be moved into PSRAM on startup. If `SPIRAM_RODATA` parameter is also enabled, the code that normally requires execution during the SPI1 flash operation does not need to be placed in IRAM, thus optimizing RAM usage. By default, this parameter is disabled.

SPIRAM_RODATA: This configuration parameter allows moving read-only data from flash to PSRAM. If `SPIRAM_FETCH_INSTRUCTIONS` parameter is also enabled, the code that normally requires execution during the SPI1 flash operation does not need to be placed in IRAM, thus optimizing RAM usage.

Installing Zephyr: A Step-by-Step Guide

To install Zephyr RTOS and the necessary tools, follow the instructions in the Getting Started Guide. By the end of the process, you will have a command-line Zephyr development environment set up and ready to build your application.

Using PSRAM for Dynamic Memory Allocation

PSRAM memory blocks can be made available to applications through Zephyr's shared multi-heap library. The shared multi-heap memory pool manager uses the multi-heap allocator to manage a set of reserved memory regions with varying capabilities and attributes. For PSRAM, enabling the `ESP_SPIRAM` and `SHARED_MULTI_HEAP` parameters causes the external PSRAM to be mapped into the data virtual memory space during Zephyr's early initialization stage. The shared multi-heap framework is initialized, and the PSRAM memory region is added to the pool.

If an application needs a memory block allocated from PSRAM, it must call `shared_multi_heap_alloc()` whith `SMH_REG_ATTR_EXTERNAL` as a parameter. This function will return an address pointing to a memory block inside PSRAM. If an aligned memory block is required, `shared_multi_heap_aligned_alloc()` should be called instead.

With the ownership of this memory block, the application is granted permission to read from and write to its addresses. Once the memory block is no longer needed, it can be returned to the pool from which it was allocated by calling `shared_multi_heap_free()` and passing the pointer to the block as a parameter.

The following sample code shows how to use Zephyr's shared multi-heap API to allocate, use, and free memory from PSRAM:

boards/esp32s3_devkitc_procpu.overlay:

```
&psram0 {
  size = <DT_SIZE_M(8)>;
};
```

prj.conf:

```
CONFIG_LOG=y
CONFIG_ESP_SPIRAM=y
```

```
CONFIG_ESP32S3_DATA_CACHE_64KB=y
CONFIG_ESP_SPIRAM_MEMTEST=y
```

src/main.c:

```
#include <zephyr/kernel.h>
#include <soc/soc_memory_layout.h>
#include <zephyr/multi_heap/shared_multi_heap.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(PSRAM_SAMPLE, LOG_LEVEL_INF);

int main(void)
{
    uint32_t *p_mem, k;

    LOG_INF("Sample started");

    p_mem = shared_multi_heap_aligned_alloc(SMH_REG_ATTR_EXTERNAL, 32, 1024*size

    if (p_mem == NULL) {
        LOG_ERR("PSRAM memory allocation failed!");
        return -ENOMEM;
    }

    for (k = 0; k < 1024; k++) {
        p_mem[k] = k;
    }

    for (k = 0; k < 1024; k++) {
        if (p_mem[k] != k) {
            LOG_ERR("p_mem[%"PRIu32"]: %"PRIu32" (expected value %"PRIu32")", k,
            break;
        }
    }

    shared_multi_heap_free(p_mem);

    if (k < 1024) {
        LOG_ERR("Failed checking memory contents.");
        return -1;
```

```
    return 0;
}
```

To build a project containing these two files targeting a `ESP32-S3-DevKitC-1` board:

```
west build -b esp32s3_devkitc/esp32s3/procpu <project folder path> --pristine
```

This command will create a directory called `build`, which will contain the binary file of our sample along with other intermediate files produced during the building process.

To flash the generated binary into the `ESP32-S3-DevKitC-1` board, run:

```
west flash
```

To open a console and see the log messages produced during the sample execution, run:

```
west espressif monitor
```

Here are the sample messages:

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fc8fa20,len:0x2178
load:0x40374000,len:0xba10
entry 0x403795b0
I (62) soc_init: ESP Simple boot
I (62) soc_init: compile time Mar 10 2025 18:43:57
W (62) soc_init: Unicore bootloader
I (62) soc_init: chip revision: v0.2
```

```
I (75) boot: DRAM: lma 0x00000020 vma 0x3fc8fa20 len 0x2178   (8568)
I (81) boot: IRAM: lma 0x000021a0 vma 0x40374000 len 0xba10   (47632)
I (88) boot: IRAM: lma 0x0000dbc8 vma 0x00000000 len 0x2430   (9264)
I (94) boot: IMAP: lma 0x00010000 vma 0x42000000 len 0x4524   (17700)
I (100) boot: IRAM: lma 0x0001452c vma 0x00000000 len 0xbacc   (47820)
I (106) boot: DMAP: lma 0x00020000 vma 0x3c010000 len 0x1674   (5748)
I (112) boot: Image with 6 segments
I (116) boot: IROM segment: paddr=00010000h, vaddr=42000000h, size=04522h ( 176
I (123) boot: DROM segment: paddr=00020000h, vaddr=3c010000h, size=01680h (  57
I (142) boot: libc heap size 336 kB.
I (142) spi_flash: detected chip: gd
I (142) spi_flash: flash io: dio
I (143) octal_psram: vendor id    : 0x0d (AP)
I (144) octal_psram: dev id       : 0x02 (generation 3)
I (149) octal_psram: density      : 0x03 (64 Mbit)
I (155) octal_psram: good-die     : 0x01 (Pass)
I (158) octal_psram: Latency      : 0x01 (Fixed)
I (163) octal_psram: VCC          : 0x01 (3V)
I (167) octal_psram: SRF          : 0x01 (Fast Refresh)
I (172) octal_psram: BurstType    : 0x01 (Hybrid Wrap)
I (176) octal_psram: BurstLen     : 0x01 (32 Byte)
I (181) octal_psram: Readlatency  : 0x02 (10 cycles@Fixed)
I (186) octal_psram: DriveStrength: 0x00 (1/1)
I (191) MSPI Timing: PSRAM timing tuning index: 6
I (195) esp_psram: Found 8MB PSRAM device
I (198) esp_psram: Speed: 80MHz
I (608) esp_psram: SPI SRAM memory test OK
*** Booting Zephyr OS build v4.0.0-6082-g6402eb6e9788 ***
[00:00:00.636,000] <inf> PSRAM_SAMPLE: Sample started
[00:00:00.636,000] <inf> PSRAM_SAMPLE: Sample finished successfully!
```

Once the sample finishes executing successfully, we can conclude that the memory allocated from PSRAM was read and written to without any issues.

Adding PSRAM to Your ESP32 Memory Map

Once `ESP_SPIRAM` is enabled, a section called `.ext_ram.bss` will be created. This section will hold non-initialized global variables that will later be placed in

boards/esp32s3_devkitc_procpu.overlay:

```
&psram0 {
  size = <DT_SIZE_M(8)>;
};
```

prj.conf:

```
CONFIG_LOG=y
CONFIG_ESP_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_SPIRAM_SPEED_80M=y
CONFIG_ESP32S3_DATA_CACHE_64KB=y
CONFIG_ESP_SPIRAM_MEMTEST=n
```

main.c:

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(PSRAM_SAMPLE, LOG_LEVEL_INF);

#define PSRAM_TEST_VECTOR_LEN (512*1024)

__attribute__ ((section (".ext_ram.bss"))) uint8_t psram_vector[PSRAM_TEST_VECT(

int main(void)
{
    uint32_t k;

    LOG_INF("Sample started");

    LOG_DBG("Writing...");
    for (k = 0; k < PSRAM_TEST_VECTOR_LEN; k++) {
        psram_vector[k] = (uint8_t)k;
        LOG_DBG("psram_vector[%"PRIu32"]: %"PRIu8, k, psram_vector[k]);
    }

    LOG_DBG("Reading...");
```

```
            LOG_ERR("Verification failed!");
            return -1;
        }
    }

    LOG_INF("Sample finished successfully!");

    return 0;
}
```

To build a project containing these two files targeting a `ESP32-S3-DevKitC-1` board, run:

```
$ west build -b esp32s3_devkitc/esp32s3/procpu <project folder path> --pristine
```

This command will create a directory called `build`, which will contain the binary file of our sample along with other intermediate files produced during the building process.

Before we move on, let's take a look inside the `build/zephyr/zephyr.map` file:

```
...
11290 *(SORT_BY_ALIGNMENT(.ext_ram.bss*))
11291 .ext_ram.bss    0x000000003c020000    0x80000 app/libapp.a(main.c.obj)
11292                 0x000000003c020000                psram_vector
11293                 0x000000003c0a0000                . = ALIGN (0x10)
11294                 0x000000003c0a0000                _ext_ram_bss_end = ABSOL
11296                 0x000000003c2a0000                . = (. + 0x200000)
11297 *fill*          0x000000003c0a0000   0x200000
11298                 0x000000003c2a0000                . = ALIGN (0x10)
11299                 0x000000003c2a0000                _ext_ram_heap_end = ABSO
11300                 0x000000003c2a0000                _ext_ram_end = ABSOLUTE
...
```

ESP32-S3. We can also see that although `SHARED_MULTI_HEAP` parameter was not explicitly enabled, it has some area reserved for `spiram_head`. It happens because `SHARED_MULTI_HEAP` parameter is enabled by default once `ESP_SPIRAM` is enabled.

Now let's flash the binary onto the `ESP32-S3-DevKitC-1` board and observe the messages from the sample:

```
$ west flash
$ west espressif monitor


ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fc8fa20,len:0x2178
load:0x40374000,len:0xba10
entry 0x403795b0
I (62) soc_init: ESP Simple boot
I (62) soc_init: compile time Mar 10 2025 18:40:57
W (62) soc_init: Unicore bootloader
I (62) soc_init: chip revision: v0.2
I (65) flash_init: Boot SPI Speed : 80MHz
I (68) flash_init: SPI Mode       : DIO
I (72) flash_init: SPI Flash Size : 8MB
I (75) boot: DRAM: lma 0x00000020 vma 0x3fc8fa20 len 0x2178   (8568)
I (81) boot: IRAM: lma 0x000021a0 vma 0x40374000 len 0xba10   (47632)
I (88) boot: IRAM: lma 0x0000dbc8 vma 0x00000000 len 0x2430   (9264)
I (94) boot: IMAP: lma 0x00010000 vma 0x42000000 len 0x4484   (17540)
I (100) boot: IRAM: lma 0x0001448c vma 0x00000000 len 0xbb6c   (47980)
I (106) boot: DMAP: lma 0x00020000 vma 0x3c010000 len 0x1654   (5716)
I (112) boot: Image with 6 segments
I (116) boot: IROM segment: paddr=00010000h, vaddr=42000000h, size=04482h ( 175
I (123) boot: DROM segment: paddr=00020000h, vaddr=3c010000h, size=01660h (  57
I (142) boot: libc heap size 336 kB.
I (142) spi_flash: detected chip: gd
I (142) spi_flash: flash io: dio
I (143) octal_psram: vendor id    : 0x0d (AP)
I (144) octal_psram: dev id       : 0x02 (generation 3)
```

```
I (163) octal_psram: VCC          : 0x01 (3V)
I (167) octal_psram: SRF          : 0x01 (Fast Refresh)
I (172) octal_psram: BurstType    : 0x01 (Hybrid Wrap)
I (176) octal_psram: BurstLen     : 0x01 (32 Byte)
I (181) octal_psram: Readlatency  : 0x02 (10 cycles@Fixed)
I (186) octal_psram: DriveStrength: 0x00 (1/1)
I (191) MSPI Timing: PSRAM timing tuning index: 6
I (195) esp_psram: Found 8MB PSRAM device
I (198) esp_psram: Speed: 80MHz
I (608) esp_psram: SPI SRAM memory test OK
*** Booting Zephyr OS build v4.0.0-6082-g6402eb6e9788 ***
[00:00:00.648,000] <inf> PSRAM_SAMPLE: Sample started
[00:00:00.683,000] <inf> PSRAM_SAMPLE: Sample finished successfully!
```

Here, we can observe that the application now starts earlier compared to the previous sample. This improvement results from disabling the `ESP_SPIRAM_MEMTEST` parameter, which bypasses the PSRAM memory test that previously took a few hundred milliseconds. in

Placing Task Stack in PSRAM

Another way to to take advantage of PSRAM is by placing Zephyr's task stacks in the `.ext_ram.bss` section via `Z_KERNEL_STACK_DEFINE_IN()` with its third parameter `lsect` being `__attribute__((section(".ext_ram.bss")))` .

> ***Note:*** *Be careful before deciding to put your task stacks in PSRAM: When flash cache is disabled (for example, if the flash is being written to), the external RAM also becomes inaccessible. Any read operations from or write operations to it will lead to an illegal cache access exception. You can find more restrictions regarding the use of external RAM here*

Following you will find a sample showing how to place a Zephyr's task stack in PSRAM:

boards/esp32s3_devkitc_procpu.overlay:

```
);
```

prj.conf:

```
CONFIG_ESP_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_SPIRAM_SPEED_80M=y
CONFIG_ESP32S3_DATA_CACHE_64KB=y
```

src/main.c:

```
#include <stdio.h>
#include <inttypes.h>
#include <zephyr/kernel.h>

#define MY_TSTACK_SIZE 1024
static Z_KERNEL_STACK_DEFINE_IN(my_tstack, MY_TSTACK_SIZE, __attribute__((sectio
static struct k_thread my_tdata;

void my_tfunc(void *arg1, void *arg2, void *arg3)
{
    uint32_t my_tcounter = 0;

    printf("my_tstack: 0x%"PRIX32"\n", (uint32_t)my_tstack);

    while(1)
    {
        printf("%"PRIu32" - Hello World! - %s\n", ++my_tcounter, CONFIG_BOARD_TAI
        k_sleep(K_MSEC(250));
    }
}

int main(void)
{
    k_tid_t tid = k_thread_create(&my_tdata, my_tstack, MY_TSTACK_SIZE,
                  my_tfunc, NULL, NULL, NULL, K_PRIO_PREEMPT(0),
                  K_INHERIT_PERMS, K_NO_WAIT);
```

To build, flash and see the sample log, type:

```
$ west build -b esp32s3_devkitc/esp32s3/procpu <application> --pristine
$ west flash
$ west espressif monitor
```

Here is the log emitted by your board:

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x2b (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fc8dc80,len:0x19f0
load:0x40374000,len:0x9c64
entry 0x40377804
I (56) soc_init: ESP Simple boot
I (57) soc_init: compile time Mar 10 2025 19:20:47
W (57) soc_init: Unicore bootloader
I (57) soc_init: chip revision: v0.2
I (59) flash_init: Boot SPI Speed : 80MHz
I (63) flash_init: SPI Mode       : DIO
I (66) flash_init: SPI Flash Size : 8MB
I (70) boot: DRAM: lma 0x00000020 vma 0x3fc8dc80 len 0x19f0   (6640)
I (76) boot: IRAM: lma 0x00001a18 vma 0x40374000 len 0x9c64   (40036)
I (82) boot: IRAM: lma 0x0000b688 vma 0x00000000 len 0x4970   (18800)
I (88) boot: IMAP: lma 0x00010000 vma 0x42000000 len 0x3de8   (15848)
I (95) boot: IRAM: lma 0x00013df0 vma 0x00000000 len 0xc208   (49672)
I (101) boot: DMAP: lma 0x00020000 vma 0x3c010000 len 0x1244   (4676)
I (107) boot: Image with 6 segments
I (110) boot: IROM segment: paddr=00010000h, vaddr=42000000h, size=03DE6h ( 158
I (118) boot: DROM segment: paddr=00020000h, vaddr=3c010000h, size=01250h (  468
I (137) boot: libc heap size 347 kB.
I (137) spi_flash: detected chip: gd
I (137) spi_flash: flash io: dio
I (137) octal_psram: vendor id    : 0x0d (AP)
I (139) octal_psram: dev id       : 0x02 (generation 3)
I (144) octal_psram: density      : 0x03 (64 Mbit)
I (149) octal_psram: good-die     : 0x01 (Pass)
```

```
I (166) octal_psram: BurstType    : 0x01 (Hybrid Wrap)
I (171) octal_psram: BurstLen     : 0x01 (32 Byte)
I (176) octal_psram: Readlatency  : 0x02 (10 cycles@Fixed)
I (181) octal_psram: DriveStrength: 0x00 (1/1)
I (187) MSPI Timing: PSRAM timing tuning index: 6
I (189) esp_psram: Found 8MB PSRAM device
I (193) esp_psram: Speed: 80MHz
I (602) esp_psram: SPI SRAM memory test OK/
*** Booting Zephyr OS build v4.0.0-6083-g3a8e95b7f490 ***
my_tstack: 0x3C020000
1 - Hello World! - esp32s3_devkitc/esp32s3/procpu
2 - Hello World! - esp32s3_devkitc/esp32s3/procpu
3 - Hello World! - esp32s3_devkitc/esp32s3/procpu
4 - Hello World! - esp32s3_devkitc/esp32s3/procpu
5 - Hello World! - esp32s3_devkitc/esp32s3/procpu
6 - Hello World! - esp32s3_devkitc/esp32s3/procpu
7 - Hello World! - esp32s3_devkitc/esp32s3/procpu
8 - Hello World! - esp32s3_devkitc/esp32s3/procpu
9 - Hello World! - esp32s3_devkitc/esp32s3/procpu
...
```

You can confirm that the task stack `my_tstack` was allocated in PSRAM because the address `0x3C020000` is a virtual address mapping an external RAM position.

Running Code from PSRAM Instead of Flash

Enabling the `SPIRAM_FETCH_INSTRUCTIONS` parameter will move instructions from flash to PSRAM during startup, and if the `ESP_SPIRAM_MEMTEST` parameter is also enabled, the code that normally requires execution during the SPI1 flash operation does not need to be placed in IRAM, thus optimizing RAM usage.

To check the effects of `SPIRAM_FETCH_INSTRUCTIONS` and `SPIRAM_RODATA` parameters, let's build `hello_world` first without enabling them and then enable them.

```
west build -b esp32s3_devkitc/esp32s3/procpu zephyr/samples/hello_world/ --prist
```

```
Memory region         Used Size  Region Size  %age Used
           FLASH:      135376 B    8388352 B      1.61%
     iram0_0_seg:       39056 B     343552 B     11.37%
     dram0_0_seg:       39072 B     327168 B     11.94%
     irom0_0_seg:       14642 B         8 MB      0.17%
     drom0_0_seg:       69840 B         8 MB      0.83%
    rtc_iram_seg:          0 GB         8 KB      0.00%
    rtc_data_seg:          0 GB         8 KB      0.00%
    rtc_slow_seg:          0 GB         8 KB      0.00%
        IDT_LIST:          0 GB         8 KB      0.00%


ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0xa (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x3fc8d8a0,len:0x176c
load:0x40374000,len:0x9880
entry 0x40377bd0
I (67) soc_init: ESP Simple boot
I (68) soc_init: compile time Dec  4 2024 19:25:19
W (68) soc_init: Unicore bootloader
I (68) spi_flash: detected chip: gd
I (70) spi_flash: flash io: dio
W (73) spi_flash: Detected size(8192k) larger than the size in the binary image
I (85) soc_init: chip revision: v0.2
I (88) flash_init: Boot SPI Speed : 40MHz
I (92) flash_init: SPI Mode       : DIO
I (96) flash_init: SPI Flash Size : 8MB
I (99) soc_random: Enabling RNG early entropy source
I (104) boot: DRAM: lma 0x00000020 vma 0x3fc8d8a0 len 0x176c   (5996)
I (110) boot: IRAM: lma 0x00001794 vma 0x40374000 len 0x9880   (39040)
I (116) boot: padd: lma 0x0000b028 vma 0x00000000 len 0x4fd0   (20432)
I (123) boot: IMAP: lma 0x00010000 vma 0x42000000 len 0x3934   (14644)
I (129) boot: padd: lma 0x0001393c vma 0x00000000 len 0xc6bc   (50876)
I (135) boot: DMAP: lma 0x00020000 vma 0x3c010000 len 0x10d0   (4304)
```

```
I (171) soc_random: Disabling RNG early entropy source
I (171) boot: Disabling glitch detection
I (171) boot: Jumping to the main image...
I (208) heap_runtime: ESP heap runtime init at 0x3fc918a0 size 351 kB.

*** Booting Zephyr OS build v4.0.0-1471-ge60f04096cd8 ***
Hello World! esp32s3_devkitc/esp32s3/procpu
```

Building `hello_world` with `SPIRAM_FETCH_INSTRUCTIONS` and `SPIRAM_RODATA` enabled

boards/esp32s3_devkitc_procpu.overlay:

```
&psram0 {
  size = <DT_SIZE_M(8)>;
};
```

```
west build -b esp32s3_devkitc/esp32s3/procpu zephyr/samples/hello_world/ --prist
```

```
Memory region         Used Size  Region Size  %age Used
           FLASH:      135916 B    8388352 B      1.62%
     iram0_0_seg:       43384 B     343552 B     12.63%
     dram0_0_seg:       45104 B     327168 B     13.79%
     irom0_0_seg:       15942 B          8 MB      0.19%
     drom0_0_seg:       70380 B          8 MB      0.84%
     ext_ram_seg:        1152 KB    8388544 B     14.06%
    rtc_iram_seg:           0 GB         8 KB      0.00%
    rtc_data_seg:           0 GB         8 KB      0.00%
    rtc_slow_seg:           0 GB         8 KB      0.00%
        IDT_LIST:           0 GB         8 KB      0.00%
```

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
```

```
I (72) soc_init: ESP Simple boot
I (72) soc_init: compile time Dec  4 2024 19:32:57
W (72) soc_init: Unicore bootloader
I (72) spi_flash: detected chip: gd
I (75) spi_flash: flash io: dio
W (78) spi_flash: Detected size(8192k) larger than the size in the binary image
I (90) soc_init: chip revision: v0.2
I (93) flash_init: Boot SPI Speed : 40MHz
I (97) flash_init: SPI Mode       : DIO
I (100) flash_init: SPI Flash Size : 8MB
I (104) soc_random: Enabling RNG early entropy source
I (109) boot: DRAM: lma 0x00000020 vma 0x3fc8e988 len 0x1c2c   (7212)
I (115) boot: IRAM: lma 0x00001c54 vma 0x40374000 len 0xa968   (43368)
I (121) boot: padd: lma 0x0000c5c8 vma 0x00000000 len 0x3a30   (14896)
I (127) boot: IMAP: lma 0x00010000 vma 0x42000000 len 0x3e48   (15944)
I (134) boot: padd: lma 0x00013e50 vma 0x00000000 len 0xc1a8   (49576)
I (140) boot: DMAP: lma 0x00020000 vma 0x3c010000 len 0x12ec   (4844)
I (146) boot: Image with 6 segments
I (149) boot: DROM segment: paddr=00020000h, vaddr=3c010000h, size=012F0h (  48
I (157) boot: IROM segment: paddr=00010000h, vaddr=42000000h, size=03E46h ( 159
I (176) soc_random: Disabling RNG early entropy source
I (176) boot: Disabling glitch detection
I (176) boot: Jumping to the main image...
I (177) octal_psram: vendor id    : 0x0d (AP)
I (181) octal_psram: dev id       : 0x02 (generation 3)
I (186) octal_psram: density      : 0x03 (64 Mbit)
I (190) octal_psram: good-die     : 0x01 (Pass)
I (195) octal_psram: Latency      : 0x01 (Fixed)
I (199) octal_psram: VCC          : 0x01 (3V)
I (203) octal_psram: SRF          : 0x01 (Fast Refresh)
I (208) octal_psram: BurstType    : 0x01 (Hybrid Wrap)
I (213) octal_psram: BurstLen     : 0x01 (32 Byte)
I (217) octal_psram: Readlatency  : 0x02 (10 cycles@Fixed)
I (223) octal_psram: DriveStrength: 0x00 (1/1)
I (227) MSPI Timing: PSRAM timing tuning index: 6
I (231) esp_psram: Found 8MB PSRAM device
I (235) esp_psram: Speed: 80MHz
I (245) mmu_psram: Instructions copied and mapped to SPIRAM
I (252) mmu_psram: Read only data copied and mapped to SPIRAM
I (652) esp_psram: SPI SRAM memory test OK
I (708) heap_runtime: ESP heap runtime init at 0x3fc93030 size 345 kB.
```

Here, we can confirm that instructions and read-only data were copied and mapped to PSRAM, which can optimize RAM usage when there is code that needs to execute during an SPI1 flash operation.

Letting ESP32 Wi-Fi Use PSRAM

Another potential use of PSRAM to reduce SRAM usage is by enabling `ESP32_WIFI_NET_ALLOC_SPIRAM` parameter, allowing the ESP32 Wi-Fi stack to dynamically allocate memory from PSRAM.

To check the results, let's build the `zephyr/samples/net/wifi/shell`, first with `ESP32_WIFI_NET_ALLOC_SPIRAM` disabled and then with this parameter enabled. In both cases let's flash the binary into the board and launch a console to interact with the shell. After connect to an access point we will execute `net allocs` to get information from where memory is being allocated in both cases.

Building `zephyr/samples/net/wifi/shell` with `ESP32_WIFI_NET_ALLOC_SPIRAM` disabled

Building, flashing, and monitoring:

```
$ west build -b esp32s3_devkitc/esp32s3/procpu zephyr/samples/net/wifi/shell --
$ west flash
$ west espressif monitor
```

Booting and interacting with the wifi shell to obtain information about the origing of the memory allocated by the wifi stack:

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
```

```
entry 0x4037c6e4
I (71) soc_init: ESP Simple boot
I (71) soc_init: compile time Mar 10 2025 18:22:27
W (71) soc_init: Unicore bootloader
I (71) soc_init: chip revision: v0.2
I (73) flash_init: Boot SPI Speed : 80MHz
I (77) flash_init: SPI Mode       : DIO
I (81) flash_init: SPI Flash Size : 8MB
I (84) boot: DRAM: lma 0x00000020 vma 0x3fc922b0 len 0x3648   (13896)
I (90) boot: IRAM: lma 0x00003670 vma 0x40374000 len 0xe294   (58004)
I (97) boot: IRAM: lma 0x00011918 vma 0x00000000 len 0xe6e0   (59104)
I (103) boot: IMAP: lma 0x00020000 vma 0x42000000 len 0x5a478  (369784)
I (109) boot: IRAM: lma 0x0007a480 vma 0x00000000 len 0x5b78   (23416)
I (115) boot: DMAP: lma 0x00080000 vma 0x3c060000 len 0x16f94  (94100)
I (122) boot: Image with 6 segments
I (125) boot: IROM segment: paddr=00020000h, vaddr=42000000h, size=5A476h (3697
I (132) boot: DROM segment: paddr=00080000h, vaddr=3c060000h, size=16FA0h ( 941
I (151) boot: libc heap size 166 kB.
I (152) spi_flash: detected chip: gd
I (152) spi_flash: flash io: dio

*** Booting Zephyr OS build v4.0.0-6082-g6402eb6e9788 ***uart:~$ wifi scan
Scan requested

Num  | SSID                                (len) | Chan (Band)    | RSSI | Security
1    | Soares                              6     | 11   (2.4GHz) | -49  | WPA2-PSK
...
uart:~$ wifi connect --key-mgmt 1 --ssid <SSID> --passphrase <PASSPHRASE>
Connection requested
Connected
[00:04:27.263,000] <inf> net_dhcpv4: Received: 192.168.15.2
uart:~$ net ping 192.168.15.17
PING 192.168.15.17
28 bytes from 192.168.15.17 to 192.168.15.2: icmp_seq=1 ttl=64 time=8 ms
28 bytes from 192.168.15.17 to 192.168.15.2: icmp_seq=2 ttl=64 time=224 ms
28 bytes from 192.168.15.17 to 192.168.15.2: icmp_seq=3 ttl=64 time=455 ms
uart:~$ net allocs
Network memory allocations

memory     Status Pool   Function alloc -> freed
0x3fcb8140  free     RX  eth_esp32_rx():139 -> net_icmpv4_input():648
```

Building `zephyr/samples/net/wifi/shell` with `ESP32_WIFI_NET_ALLOC_SPIRAM` enabled

Building, flashing and monitoring:

socs/esp32s3_procpu.overlay:

```
...
&psram0 {
  size = <DT_SIZE_M(8)>;
};
```

```
$ west build -b esp32s3_devkitc/esp32s3/procpu zephyr/samples/net/wifi/shell --p
$ west flash
$ west espressif monitor
```

Booting and interacting with the wifi shell to obtain information about the origing of the memory allocated by the wifi stack:

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fc93140,len:0x3a38
load:0x40374000,len:0xf130
entry 0x4037c85c
I (73) soc_init: ESP Simple boot
I (73) soc_init: compile time Mar 10 2025 18:05:09
W (74) soc_init: Unicore bootloader
I (74) soc_init: chip revision: v0.2
I (76) flash_init: Boot SPI Speed : 80MHz
I (80) flash_init: SPI Mode       : DIO
I (83) flash_init: SPI Flash Size : 8MB
I (87) boot: DRAM: lma 0x00000020 vma 0x3fc93140 len 0x3a38   (14904)
```

```
I (112) boot: IRAM: lma 0x0007a92c vma 0x00000000 len 0x56cc   (22220)
I (118) boot: DMAP: lma 0x00080000 vma 0x3c060000 len 0x1714c  (94540)
I (124) boot: Image with 6 segments
I (127) boot: IROM segment: paddr=00020000h, vaddr=42000000h, size=5A922h (3709
I (135) boot: DROM segment: paddr=00080000h, vaddr=3c060000h, size=17150h ( 945
I (154) boot: libc heap size 182 kB.
I (154) spi_flash: detected chip: gd
I (154) spi_flash: flash io: dio
I (155) octal_psram: vendor id    : 0x0d (AP)
I (156) octal_psram: dev id       : 0x02 (generation 3)
I (161) octal_psram: density      : 0x03 (64 Mbit)
I (166) octal_psram: good-die     : 0x01 (Pass)
I (170) octal_psram: Latency      : 0x01 (Fixed)
I (174) octal_psram: VCC          : 0x01 (3V)
I (179) octal_psram: SRF          : 0x01 (Fast Refresh)
I (183) octal_psram: BurstType    : 0x01 (Hybrid Wrap)
I (189) octal_psram: BurstLen     : 0x01 (32 Byte)
I (193) octal_psram: Readlatency  : 0x02 (10 cycles@Fixed)
I (198) octal_psram: DriveStrength: 0x00 (1/1)
I (203) MSPI Timing: PSRAM timing tuning index: 6
I (207) esp_psram: Found 8MB PSRAM device
I (210) esp_psram: Speed: 80MHz
I (619) esp_psram: SPI SRAM memory test OK

*** Booting Zephyr OS build v4.0.0-6082-g6402eb6e9788 ***
uart:~$ wifi scan
Scan requested

Num  | SSID                                (len) | Chan (Band)    | RSSI | Security
1    | Soares                              6     | 11   (2.4GHz) | -63  | WPA2-PSK
...
uart:~$ wifi connect --key-mgmt 1 --ssid <SSID> --passphrase <PASSPHRASE>
Connection requested
Connected
[00:01:31.902,000] <inf> net_dhcpv4: Received: 192.168.15.2
uart:~$ net ping 192.168.15.17
PING 192.168.15.17
28 bytes from 192.168.15.17 to 192.168.15.2: icmp_seq=1 ttl=64 time=231 ms
28 bytes from 192.168.15.17 to 192.168.15.2: icmp_seq=2 ttl=64 time=6 ms
28 bytes from 192.168.15.17 to 192.168.15.2: icmp_seq=3 ttl=64 time=338 ms
uart:~$ net allocs
```

```
0x3c0839a0  free     RX  eth_esp32_rx():139 -> processing_data():178
0x3c083720  free     TX  icmpv4_handle_echo_request():446 -> ethernet_send():804
0x3c0828dc  free  TDATA  ethernet_fill_header():608 -> ethernet_send():804
```

Examining the output from both sessions, we can confirm that in the first case, network stack allocations were made using memory from internal RAM – addresses `0x3fcb8140`, `0x3fcb7ec0`, `0x3fcb707` and `0x3fcb7098`. We can also confirm that in the second case, network stack allocations were taking memory from PSRAM – addresses `0x3c0839a0`, `0x3c083720`, and `0x3c0828dc` – thus proving that enabling `ESP32_WIFI_NET_ALLOC_SPIRAM` avoids allocating memory from the precious internal SRAM and takes advantage of PSRAM.

Final Thoughts

Throughout this article, we explored three different strategies for utilizing PSRAM: using it for dynamic memory allocation, adding it to your ESP32 memory map, and running code from PSRAM instead of Flash.

In addition to these three strategies, it is also possible to leverage PSRAM to execute external code compiled as Position Independent Code (PIC), which can be placed in PSRAM by a loader for execution. You can find more information about this way of using PSRAM by consulting Zephyr's document LLEXT – Linkable Loadable Extensions.

References

- Zephyr on ESP Devices
- SPI Flash and External SPI RAM Configuration
- Support for External RAM
  - ESP32
  - ESP32-S2

- Zephyr Project Official Site
- Zephyr Project Documentation Site
  - Getting Started Guide
  - Supported Boards and Shields
    - Espressif
      - ESP32-DevKitC-WROVER
      - ESP32-S2-DevKitC
      - ESP32-S3-DevKitC-1
  - Shared Multi Heap
  - Code And Data Relocation
  - Linkable Loadable Extensions

0 reactions

3 comments · 2 replies – *powered by giscus*

Oldest Newest

Related





30 October 2024 ·
Updated: 26 September 2025 ·
15 mins

Esp32 ESP32-S2 ESP32-S3 ESP32-C3 ESP32-C6 ESP-IDF Zephyr

This article explains the booting process of Espressif devices up to ESP32-C6 (and probably newer devices).

20 August 2024 ·
Updated: 25 September 2025 ·
11 mins

Esp32 ESP32-S2 ESP32-S3 ESP32-C3 ESP32-C6 Zephyr Memory

This article addresses the complexity of Espressif SoCs by offering a visual, structured compilation of crucial technical data from their datasheets and TRMs. The main takeaway for the reader is a simplified, actionable understanding of the memory maps, caches, peripherals, and eFuse blocks, which is essential for low-level system and driver development.



How to Create an UI Application for the ESP32-S3

2 October 2024 · 9 mins

Esp32 ESP-IDF GUI Embedded Wizard ESP32-S3 ESP32-S2